

Componentes en interfaces gráficas Java (Swing)

El desarrollo de interfaces gráficas en Java Swing se apoya de forma directa en el **uso de componentes reutilizables** como elemento central de construcción de la interfaz. Una aplicación Swing no se diseña como una única estructura monolítica, sino como un **conjunto organizado de componentes**, cada uno con una función específica dentro de la interfaz.

El programador construye la interfaz combinando componentes ya existentes —como botones, etiquetas o campos de texto— y, cuando es necesario, creando **componentes personalizados** que encapsulan comportamiento, apariencia y lógica común. Este enfoque permite estructurar la interfaz de forma clara, mantener el código organizado y facilitar su reutilización en diferentes ventanas o aplicaciones.

En Swing, todos los elementos visuales que forman una interfaz gráfica pueden considerarse componentes: desde los controles más simples hasta paneles completos que agrupan otros componentes. El uso correcto de estos componentes es clave para:

- Construir interfaces coherentes
- Evitar duplicación de código
- Facilitar el mantenimiento de la aplicación
- Separar la lógica de la interfaz del resto del programa

El desarrollo basado en componentes en Swing no se limita al uso de los componentes estándar de la biblioteca, sino que permite **extenderlos, especializarlos y combinarlos** para crear nuevos componentes adaptados a las necesidades de cada aplicación.

A lo largo de este apartado se analizará:

- El uso de los componentes Swing como base
- La creación de componentes personalizados mediante herencia
- La definición de propiedades y eventos
- La reutilización de componentes en herramientas de diseño visual como WindowBuilder

Todo ello desde un enfoque práctico orientado a la creación de interfaces gráficas bien estructuradas y reutilizables.

2.1. Componentes Swing como base

Swing proporciona un amplio conjunto de **componentes visuales estándar**, que pueden utilizarse directamente o servir como base para crear componentes personalizados.

Algunos de los componentes más habituales son:

- **JButton**
Representa un botón interactivo que permite ejecutar una acción cuando el usuario hace clic.
- **JLabel**
Muestra texto o imágenes informativas. No es interactivo, pero es fundamental para estructurar la interfaz.
- **TextField**
Permite la introducción de texto en una sola línea por parte del usuario.
- **TextArea**
Similar a **TextField**, pero permite múltiples líneas de texto.
- **JPanel**
Contenedor genérico que permite agrupar otros componentes y organizar la interfaz mediante distintos gestores de diseño (layouts).
- **CheckBox**, **RadioButton**, **ComboBox**, **Table**, etc.

Todos estos componentes:

- Heredan directa o indirectamente de **Component**
- Disponen de propiedades configurables
- Soportan eventos
- Pueden integrarse en herramientas visuales como WindowBuilder

Estos componentes constituyen la **base del sistema de componentes de Swing**.

2.2. Concepto de componente personalizado en Java

Un **componente personalizado** en Java es un componente creado por el desarrollador a partir de uno existente, adaptándolo a las necesidades específicas de una aplicación.

Herencia de componentes existentes

La forma habitual de crear un componente personalizado en Swing es mediante **herencia**, extendiendo una clase ya existente:

- Extender **Button** para crear un botón con comportamiento o estilo específico
- Extender **TextField** para validar datos
- Extender **Panel** para crear componentes compuestos (formularios, tarjetas, paneles reutilizables)

Ejemplo conceptual:

- Un botón de “Aceptar” con icono, color y validación predefinidos
- Un campo de texto que solo acepte números
- Un panel de login reutilizable

Especialización de comportamiento y apariencia

Un componente personalizado puede especializarse en dos niveles:

Apariencia: Colores, Tipografías, Bordes, Tamaños.

Comportamiento: Validación de datos, Acciones automáticas, Respuesta personalizada a eventos, Lógica encapsulada.

De este modo, el componente **encapsula tanto la estética como la funcionalidad específica**, facilitando su reutilización.

2.3. Propiedades de un componente

Atributos internos

Los componentes mantienen un **estado interno** mediante atributos (variables de instancia), que normalmente son privados y no accesibles directamente desde el exterior.

Estos atributos permiten al componente mantener información propia sin exponer detalles de implementación.

Propiedades públicas (getters y setters)

Las **propiedades** son atributos que se exponen mediante métodos públicos siguiendo el patrón estándar de Java:

- `getPropiedad()` o `isPropiedad()`
- `setPropiedad(valor)`

Ejemplo:

- `getColor(), setColor(Color c)`
- `isValid(), setValid(boolean v)`

Las propiedades:

- Forman parte de la **interfaz pública del componente**
- Pueden ser detectadas automáticamente por los IDEs (introspección)
- **Se muestran en la ventana de propiedades de herramientas como WindowBuilder**

Este mecanismo permite:

- Configurar el componente visualmente
- Integrarlo fácilmente en diseñadores gráficos
- Mantener independencia entre diseño y código

2.4. Eventos y comportamiento del componente

Gestión de eventos en Java Swing

En Java Swing, el comportamiento de los componentes se gestiona mediante un **modelo basado en eventos y listeners**.

Cuando el usuario interactúa con un componente (clic, tecla, foco, etc.), se genera un **evento**, que es capturado por uno o varios **listeners** asociados al componente.

En Swing existen **dos formas habituales y fundamentales** de gestionar eventos, que son las que se emplean en la mayoría de interfaces gráficas.

Opción 1: Implementar el listener con **implements**

En esta opción, la **clase que contiene la interfaz gráfica implementa directamente la interfaz del listener**, por ejemplo `ActionListener`.

Características

- La clase implementa la interfaz del evento
- El método del listener se define una sola vez
- Los componentes se registran usando `addActionListener(this)`

Ejemplo

```
public class VentanaPrincipal extends JFrame implements ActionListener {
    private JButton btnReconocedor;
    public VentanaPrincipal() {
        btnReconocedor = new JButton("Reconocedor");
        btnReconocedor.addActionListener(this);
        add(btnReconocedor);
        setSize(300, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == btnReconocedor) {
            iniciarReconocedor();
        }
    }
    private void iniciarReconocedor() {
        System.out.println("Reconocedor iniciado");
    }
}
```

Ventajas

- Código fácil de entender
- Adecuado para interfaces sencillas
- Útil para aprendizaje inicial

Inconvenientes

- El método `actionPerformed()` puede crecer demasiado
- Es necesario identificar el origen del evento (`getSource()`)
- Menor encapsulación del comportamiento

Opción 2: Gestionar el evento directamente en el código (sin `implements`)

En esta opción **no se implementa la interfaz en la clase**, sino que se asocia el listener directamente al componente mediante una **clase anónima**.

Es una forma muy habitual en Swing y **más directa** para gestionar eventos simples.

Ejemplo (tal y como se usa en la práctica)

```
JButton btnNewButton_1 = new JButton("Reconocedor");
btnNewButton_1.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        iniciarReconocedor();
    }
});
```

Características

- El listener se define junto al componente
- No es necesario usar `implements`
- El código del evento queda localizado

Ventajas

- Código más claro y compacto
- No es necesario comprobar el origen del evento
- Mejor organización cuando hay muchos botones
- Muy utilizado en aplicaciones reales

Inconvenientes

- Si el código del evento es muy largo, puede perder legibilidad
- No es reutilizable directamente

Componentes Swing y eventos asociados

Componente Swing	Eventos principales	Listener asociado	Descripción del uso habitual
JButton	ActionEvent	ActionListener	Se dispara al hacer clic en el botón o al activarlo con teclado
	MouseEvent	MouseListener, MouseMotionListener	Detecta clics, entrada/salida del ratón, pulsaciones
	FocusEvent	FocusListener	Detecta cuando el botón gana o pierde el foco
	KeyEvent	KeyListener	Detecta pulsaciones de teclado cuando el botón tiene foco
JTextField	ActionEvent	ActionListener	Se dispara al pulsar Enter dentro del campo
	KeyEvent	KeyListener	Permite validar caracteres introducidos
	FocusEvent	FocusListener	Validaciones al ganar o perder foco
	DocumentEvent	DocumentListener	Detecta cambios en el contenido del texto
JTextArea	KeyEvent	KeyListener	Control de entrada de texto
	FocusEvent	FocusListener	Gestión de foco
	DocumentEvent	DocumentListener	Cambios en el contenido del texto
JLabel	MouseEvent	MouseListener	Puede usarse como elemento clicable
	FocusEvent	FocusListener	En interfaces accesibles

JCheckBox	ActionEvent	ActionListener	Detecta cambios de estado
	ItemEvent	ItemListener	Detecta selección / desección
JRadioButton	ActionEvent	ActionListener	Cambio de selección
	ItemEvent	ItemListener	Gestión de estado seleccionado
JComboBox	ActionEvent	ActionListener	Cambio de selección
	ItemEvent	ItemListener	Elemento seleccionado / deseleccionado
JPanel	MouseEvent	MouseListener, MouseMotionListener	Interacciones con el panel
	ComponentEvent	ComponentListener	Cambios de tamaño o visibilidad
JTable	MouseEvent	MouseListener	Selección de filas/celdas
	KeyEvent	KeyListener	Navegación por teclado
	ListSelectionEvent	ListSelectionListener	Cambio de selección
JFrame / JDialog	WindowEvent	WindowListener	Apertura, cierre, minimización
	ComponentEvent	ComponentListener	Redimensionado, movimiento
	KeyEvent	KeyListener	Atajos de teclado

Relación Evento – Listener – Método en Java Swing

Evento	Listener asociado	Método que maneja el evento
ActionEvent	ActionListener	actionPerformed(ActionEvent e)
MouseEvent	MouseListener	mouseClicked(MouseEvent e)
		mousePressed(MouseEvent e)
		mouseReleased(MouseEvent e)
		mouseEntered(MouseEvent e)
		mouseExited(MouseEvent e)
MouseEvent	MouseMotionListener	mouseMoved(MouseEvent e)
		mouseDragged(MouseEvent e)
KeyEvent	KeyListener	keyPressed(KeyEvent e)
		keyReleased(KeyEvent e)
		keyTyped(KeyEvent e)
FocusEvent	FocusListener	focusGained(FocusEvent e)
		focusLost(FocusEvent e)
ItemEvent	ItemListener	itemStateChanged(ItemEvent e)

WindowEvent	WindowListener	windowOpened(WindowEvent e)
		windowClosing(WindowEvent e)
		windowClosed(WindowEvent e)
		windowIconified(WindowEvent e)
		windowDeiconified(WindowEvent e)
		windowActivated(WindowEvent e)
		windowDeactivated(WindowEvent e)
ComponentEvent	ComponentListener	componentResized(ComponentEvent e)
		componentMoved(ComponentEvent e)
		componentShown(ComponentEvent e)
		componentHidden(ComponentEvent e)
ListSelectionEvent	ListSelectionListener	valueChanged(ListSelectionEvent e)
DocumentEvent	DocumentListener	insertUpdate(DocumentEvent e)
		removeUpdate(DocumentEvent e)
		changedUpdate(DocumentEvent e)

2.5. Creación de un componente propio a partir de Swing

Extensión de JButton / JTextField

Lo más común y sencillo la hora de crear un componente propio es crearlo extendiendo directamente componentes básicos, es decir, heredando un componente base e introducir los cambios / especificaciones que deseamos incluir en ese elemento.

Casos habituales:

- Extender `JButton` para crear botones con estilo corporativo
- Extender `JTextField` para validaciones automáticas
- Extender `JPanel` para crear componentes compuestos

Al extender un componente:

- Se pueden redefinir constructores
- Se pueden añadir propiedades propias
- Se puede modificar el comportamiento por defecto

2.6. Uso de componentes personalizados en WindowBuilder (Eclipse)

Integración en el diseñador visual

Los componentes personalizados:

- Pueden añadirse a la paleta de componentes
- Se arrastran y sueltan sobre el diseño
- Se comportan como cualquier componente estándar

Edición de propiedades desde el IDE

Gracias a los getters y setters:

- Las propiedades del componente aparecen en el panel de propiedades
- Pueden configurarse sin escribir código
- Los cambios se reflejan automáticamente en la interfaz

Esto facilita:

- El diseño visual
- La experimentación
- La separación entre diseño y lógica

EJEMPLO - Pasos para construir el componente `TextFieldDNI`

Paso 1: Decidir la clase base

Como queremos un campo de texto, partimos de:

- `JTextField` (no `TextField`, que es de AWT)

Paso 2: Definir el comportamiento añadido

- Validar el DNI cada vez que cambie el texto
- Pintar el fondo según estado: vacío / inválido / válido

Paso 3: Añadir propiedades configurables

- `emptyColor`, `invalidColor`, `validColor`
- Getters y setters para editarlos desde código (y desde WindowBuilder si lo detecta como propiedad)

Paso 4: Registrar el evento adecuado

Para detectar cambios del texto en Swing, en este caso utilizaremos `KeyListener`.

Paso 5: Empaquetar y usar

- Crear la clase en tu proyecto
- En WindowBuilder: usar "Choose component..." o añadirlo desde el classpath y arrastrar si lo detecta

```

import java.awt.Color;
import javax.swing.JTextField;
import java.awt.event.KeyEvent;
import java.awt.event.KeyListener;

public class TextFieldDNI extends JTextField implements KeyListener {
    private static final long serialVersionUID = 1L;
    private Color emptyColor = Color.LIGHT_GRAY;
    private Color invalidColor = Color.RED;
    private Color validColor = Color.GREEN;

    public TextFieldDNI() {
        super();
        setBackground(emptyColor);
        addKeyListener(this);
    }

    // --- Métodos del KeyListener ---
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        validarYActualizar();
    }

    // --- Lógica de validación y pintado ---
    private void validarYActualizar() {
        String text = getText().trim();
        if (text.isEmpty()) {
            setBackground(emptyColor);
        } else if (isValidDNI(text)) {
            setBackground(validColor);
        } else {
            setBackground(invalidColor);
        }
    }

    private boolean isValidDNI(String dni) {
        if (!dni.matches("^\\d{8}[A-Za-z]$")) return false;

        String letras = "TRWAGMYFPDXBNJZSQVHLCKE";
        int numero = Integer.parseInt(dni.substring(0, 8));
        char letra = Character.toUpperCase(dni.charAt(8));

        return letra == letras.charAt(numero % 23);
    }

    // --- Getters y setters ---
    public Color getEmptyColor() { return emptyColor; }
    public void setEmptyColor(Color emptyColor) { this.emptyColor = emptyColor; validarYActualizar(); }

    public Color getInvalidColor() { return invalidColor; }
    public void setInvalidColor(Color invalidColor) { this.invalidColor = invalidColor; validarYActualizar(); }

    public Color getValidColor() { return validColor; }
    public void setValidColor(Color validColor) { this.validColor = validColor; validarYActualizar(); }}

```

SIN IMPLEMENTS (INTERFACES) - addKeyListener(new KeyListener())

```
import java.awt.Color;
import java.awt.event.KeyEvent;
import java.awt.event.KeyListener;
import javax.swing.JTextField;

public class TextFieldDNI extends JTextField {
    private static final long serialVersionUID = 1L;
    private Color emptyColor = Color.LIGHT_GRAY;
    private Color invalidColor = Color.RED;
    private Color validColor = Color.GREEN;

    public TextFieldDNI() {
        super();
        setBackground(emptyColor);

        addKeyListener(new KeyListener() {
            @Override
            public void keyTyped(KeyEvent e) {
                validarYActualizar();
            }
            @Override
            public void keyPressed(KeyEvent e) { }

            @Override
            public void keyReleased(KeyEvent e) { }
        });
    }

    private void validarYActualizar() {
        String text = getText().trim();
        if (text.isEmpty()) {
            setBackground(emptyColor);
        } else if (isValidDNI(text)) {
            setBackground(validColor);
        } else {
            setBackground(invalidColor);
        }
    }

    private boolean isValidDNI(String dni) {
        if (!dni.matches("^\\d{8}[A-Za-z]$")) return false;
        String letras = "TRWAGMYFPDXBNJZSQVHLCKE";
        int numero = Integer.parseInt(dni.substring(0, 8));
        char letra = Character.toUpperCase(dni.charAt(8));
        return letra == letras.charAt(numero % 23);
    }

    // --- Getters y setters ---
    public Color getEmptyColor() { return emptyColor; }
    public void setEmptyColor(Color emptyColor) { this.emptyColor = emptyColor; validarYActualizar(); }

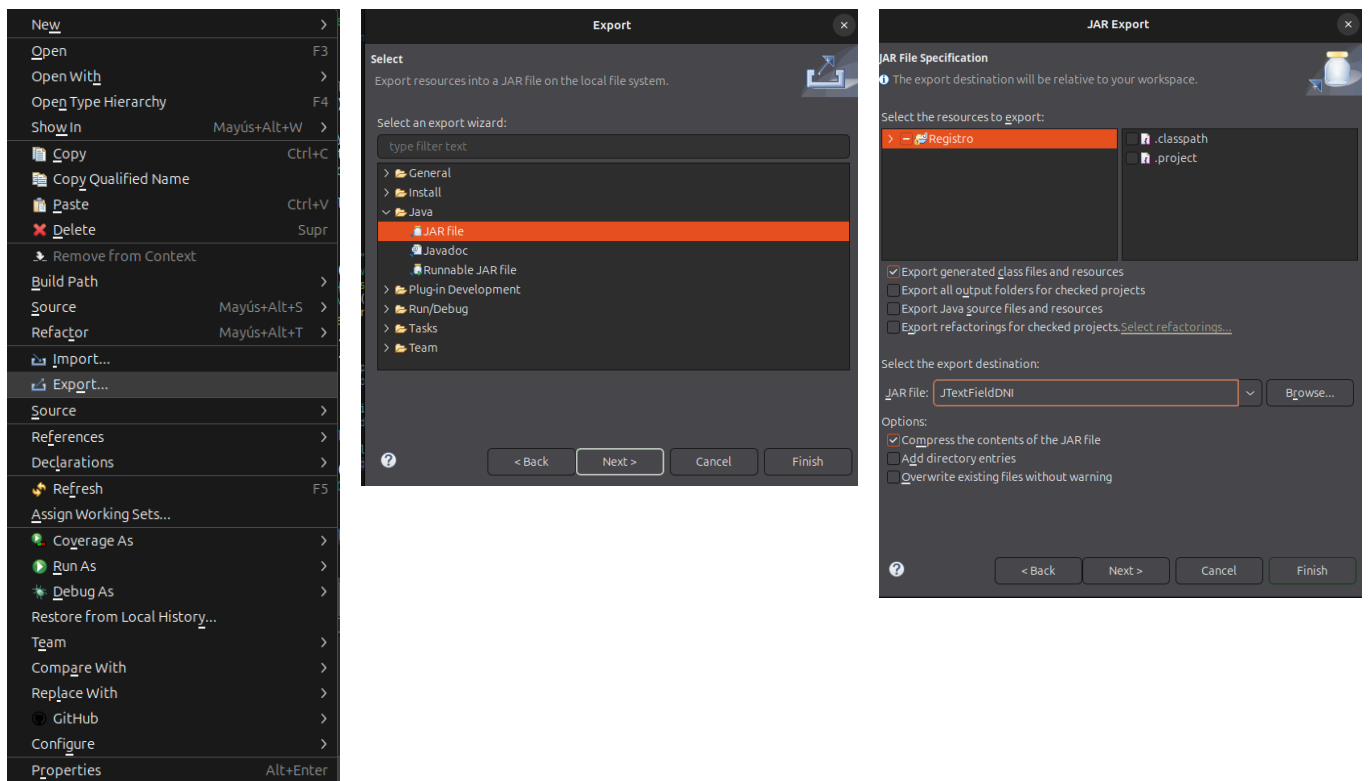
    public Color getInvalidColor() { return invalidColor; }
    public void setInvalidColor(Color invalidColor) { this.invalidColor = invalidColor; validarYActualizar(); }

    public Color getValidColor() { return validColor; }
    public void setValidColor(Color validColor) { this.validColor = validColor; validarYActualizar(); }
}
```

Paso 5: Empaquetar y usar

PASO1: Generar .jar

Sobre el fichero .java de nuestro componente pulsamos click derecho y seleccionamos export



2. En el código donde queremos utilizar el componente debemos importar el componente. En la pestaña de “diseño” seleccionamos **“Palette manager”**. Podemos añadir una categoría en donde situaremos los componentes propios e importar el jar previamente creado, al importar el jar seleccionaremos la clase correspondiente al componente que importamos.

